

Question 2

Q2(B) – Technical Support Ticket Navigator

A software company maintains active technical-support tickets dynamically. Every ticket contains

Ticket_ID and Priority, where priority is between 1 and 5. The support engineer works with one ticket

at a time and must be able to move both forward and backward through the active tickets.

New tickets must always follow this business rule: higher-priority tickets appear before lower-priority

tickets; if two tickets have the same priority, the ticket received earlier remains first.

Develop a dynamically allocated program supporting these commands:

NEW id priority – Create a new ticket and place it in its correct position.

NEXT – Move the engineer to the next ticket, if one exists.

PREVIOUS – Move the engineer to the previous ticket, if one exists.

RESOLVE id – Remove the specified ticket from the active ticket collection.

CHANGE id newPriority – Change the priority of the specified ticket and reposition it so that all

business rules remain satisfied.

SHOW – Display all active tickets from first to last as Ticket_ID and Priority.

SHOW_REVERSE – Display all active tickets from last to first.

CURRENT – Display the ticket currently selected by the engineer.

Additional Programming Requirement

Maintain a variable that counts the number of link modifications performed during every NEW,

RESOLVE, and CHANGE operation. After each such operation, display the number of link modifications.

At the end of execution, display the total number of link modifications.

The program must correctly handle:

☐ initially empty collection and only one ticket

- ❑ highest- and lowest-priority insertion
- ❑ several tickets with the same priority
- ❑ resolving the first, last, or current ticket
- ❑ changing a ticket from low to high priority and high to low priority
- ❑ invalid Ticket_ID
- ❑ movement beyond the first or last ticket

CODE:

```
#include <bits/stdc++.h>
using namespace std;

class Node{
public:
    int data;
    Node* next;
    Node* prev;
    Node(int val){
        data=val;
        next=NULL;
        prev=NULL;
    }
};

class ListNode{
public:
    Node* head;
    Node* tail;
    ListNode(){
        head=NULL;
        tail=NULL;
    }
    void push_front(int val){ // push from front
        Node* newNode=new Node(val);
        if(head==NULL){
            head=newNode;
            tail=newNode;
            return;
        }
        newNode->next=head;
        head->prev=newNode;
        head=newNode;
    }
    void push_back(int val){ // push from back
        Node* newNode= new Node(val);
        if(head==NULL){
            head=newNode;
            tail=newNode;
            return;
        }
        tail->next=newNode;
        newNode->prev=tail;
        tail=newNode;
    }
};
```

```

C: #include<bits/stdc++.h> Untitled-1
4  class Node{
16      class ListNode{
17      public:
36      void push_back(int val){ // push from back
37          Node* newNode= new Node(val);
38          if(head==NULL){
39              head=tail=newNode;
40          }
41          else{
42              newNode->prev=tail;
43              tail->next=newNode;
44              tail=newNode;
45          }
46      }
47      void pop_head(){ // pop from head
48          if(head==NULL){
49              cout<<"List is empty"<<endl;
50          }
51          else{
52              Node* temp=head;
53              head=head->next;
54              temp->next=NULL;
55              temp->next->prev=NULL;
56              delete temp;
57          }
58      }
59  }
60
61      void remove_at(int val){ // remove specific tickit id
62          if(head==NULL){
63              cout<<"List is empty"<<endl;
64          }
65          else{
66              Node* curr= head;
67              if(curr->data == val){
68                  pop_head();
69              }

```

```

C: #include<bits/stdc++.h> Untitled-1
60  class Node{
61      void remove_at(int val){ // remove specific tickit id
62          if(head==NULL){
63              cout<<"List is empty"<<endl;
64          }
65          else{
66              Node* curr= head;
67              if(curr->data == val){
68                  pop_head();
69              }
70
71              while( curr->next!=NULL && curr->next->data!=val ){
72                  curr=curr->next;
73              }
74              if(curr->next ==NULL){
75                  cout<<"Value not Found"<<endl;
76              }
77              else{
78                  curr->next->next->prev=curr;
79                  curr=curr->next;
80                  curr->prev->next=curr->next;
81                  curr->next=NULL;
82                  curr->prev=NULL;
83                  delete curr;
84              }
85          }
86      }
87
88      void addValue(int val,int prio){ // add value according to the priority
89          if(prio < 3){
90              push_front(val);
91          }
92          else{
93              push_back(val);
94          }
95      }

```

/

```
#include<bits/stdc++.h>
using namespace std;

class Node{
public:
    int data;
    Node* next;
    Node* prev;

    void addValue(int val,int prio){ // add value according to the priority
        if(prio < 3){
            push_front(val);
        }
        else{
            push_back(val);
        }
    }

    void changePrio(int val,int prio){ // change priority and rearrange
        if(head==NULL){
            cout<<"List is empty"<<endl;
        }
        else{
            Node* curr= head;

            while( curr->next!=NULL && curr->next->data!=val ){
                curr=curr->next;
            }
            if(curr->next ==NULL){
                cout<<"Value not Found"<<endl;
            }
            else{
                //for delete that node first;
                curr->next->prev=curr;
                curr=curr->next;
                curr->prev->next=curr->next;
                curr->next=NULL;
                curr->prev=NULL;
                delete curr;
            }
            //Then for re arrange the node:
        }
    }
};
```

```

void changePrio(int val,int prio){ // change priority and rearrange
    if(head==NULL){
        cout<<"List is empty"<<endl;
    }
    else{
        Node* curr= head;

        while( curr->next!=NULL && curr->next->data!=val ){
            curr=curr->next;
        }
        if(curr->next ==NULL){
            cout<<"Value not Found"<<endl;
        }
        else{
            //for delete that node first;
            curr->next->prev=curr;
            curr=curr->next;
            curr->prev->next=curr->next;
            curr->next=NULL;
            curr->prev=NULL;
            delete curr;

            //Then for re arrange the node;
            if(prio < 3){
                push_front(val);
            }
            else{
                push_back(val);
            }
        }
    }
}

void display(){ // display forward direction
    Node* temp=head;
    while(temp!=NULL){
        cout<<temp->data<<" <-> ";
        temp=temp->next;
    }
    cout<<"NULL"<<endl;
}

void display_rev(Node* head){ // reverse print
    Node* temp=head;
    if(temp==NULL){
        return;
    }
    display_rev(temp->next);
    cout<<temp->data<<" <-> ";
}

void current(){ // current Node
    Node* temp = head;
    cout<<temp->data;
}

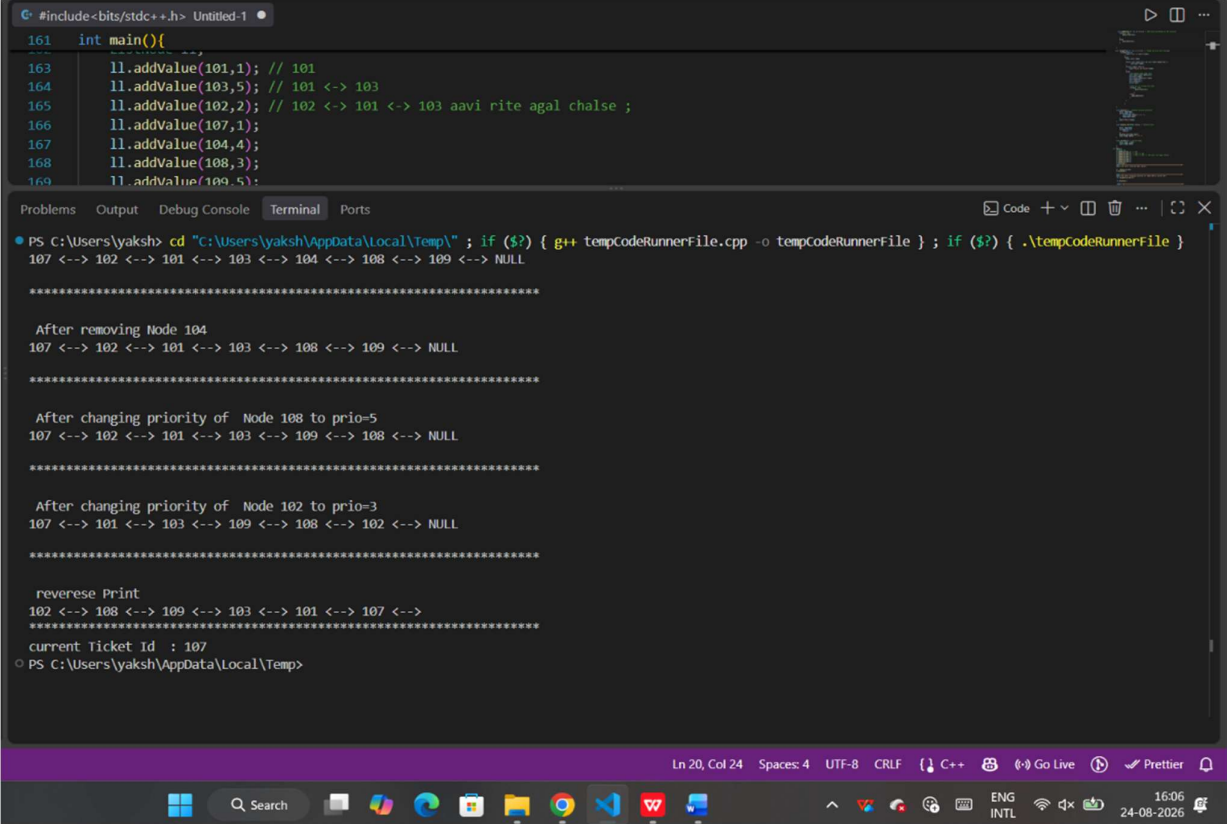
int main(){
    ListNode ll;
    ll.addValue(101,1); // 101
    ll.addValue(103,5); // 101 <-> 103
    ll.addValue(102,2); // 102 <-> 101 <-> 103 aavi rite agal chalse ;
    ll.addValue(107,1);
    ll.addValue(104,4);
    ll.addValue(108,3);
    ll.addValue(109,5);
}

```

```
#include<bits/stdc++.h> Untitled-1
161 int main(){
162     // ...
163     ll.addValue(101,1); // 101
164     ll.addValue(103,5); // 101 <-> 103
165     ll.addValue(102,2); // 102 <-> 101 <-> 103 aavi rite agal chalse ;
166     ll.addValue(107,1);
167     ll.addValue(104,4);
168     ll.addValue(108,3);
169     ll.addValue(109,5);
170     ll.display();
171     cout<<"\n*****\n";
172     cout<<"\n After removing Node 104\n";
173
174     ll.remove_at(104);
175     ll.display();
176
177     cout<<"\n*****\n";
178     cout<<"\n After changing priority of Node 108 to prio=5 \n";
179     ll.changePrio(108,5);
180
181     ll.display();
182
183     cout<<"\n*****\n";
184     cout<<"\n After changing priority of Node 102 to prio=3\n";
185     ll.changePrio(102,3);
186
187     ll.display();
188
189     cout<<"\n*****\n";
190     cout<<"\n reverse Print \n";
191
192     ll.display_rev(ll.head);
193
194     cout<<"\n*****\n";
195
196     cout<<"current Ticket Id : "<< ll.head->data<<endl;
197
198 }
```

OUTPUT :

Given in next page;



```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* head = NULL;
Node* tail = NULL;

void addValue(int val, int prio) {
    Node* newNode = new Node;
    newNode->data = val;
    if (head == NULL) {
        head = tail = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        tail = newNode;
    }
}

void printList() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    addValue(101, 1); // 101
    addValue(103, 5); // 101 <-> 103
    addValue(102, 2); // 102 <-> 101 <-> 103 aavi rite agal chalse ;
    addValue(107, 1);
    addValue(104, 4);
    addValue(108, 3);
    addValue(109, 5);

    printList();

    // Removing Node 104
    Node* temp = head;
    Node* prev = NULL;
    while (temp != NULL) {
        if (temp->data == 104) {
            if (prev == NULL) {
                head = temp->next;
            } else {
                prev->next = temp->next;
            }
            delete temp;
            temp = temp->next;
        } else {
            prev = temp;
            temp = temp->next;
        }
    }

    printList();

    // Changing priority of Node 108 to prio=5
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data == 108) {
            temp->data = 108;
            temp->prio = 5;
        }
        temp = temp->next;
    }

    printList();

    // Changing priority of Node 102 to prio=3
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data == 102) {
            temp->data = 102;
            temp->prio = 3;
        }
        temp = temp->next;
    }

    printList();

    // Reverse Print
    Node* temp = head;
    Node* prev = NULL;
    Node* next = NULL;
    while (temp != NULL) {
        next = temp->next;
        temp->next = prev;
        prev = temp;
        temp = next;
    }

    printList();

    current Ticket Id : 107
    PS C:\Users\yaksh\AppData\Local\Temp>
```

Ln 20, Col 24 Spaces: 4 UTF-8 CRLF C++ Go Live Prettier

16:06 24-08-2026